



CVRP: Método Exacto vs. Particle Swarm Optimization

Trabajo de investigación — Informes 1 y 2

Cristopher Angulo Ahumada

Optimización en Ingeniería ■ Profs. Dra. Mónica Villanueva I. y Dr. Mario Inostroza P.

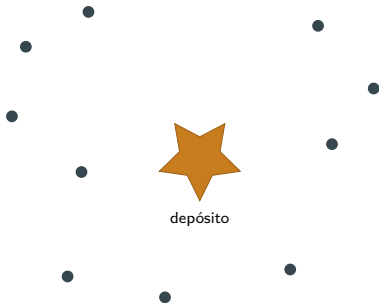
Julio de 2026

Método exacto: Laporte & Nobert (1983) + cortes RCI ■ Metaheurística: PSO con representación SR-1 (Ai & Kachitvichyanukul, 2009)

- 1 El problema: CVRP
- 2 Informe 1: método exacto
- 3 Informe 2: PSO para el CVRP
- 4 Resultados y comparación
- 5 Conclusiones
- 6 Referencias

El problema: CVRP

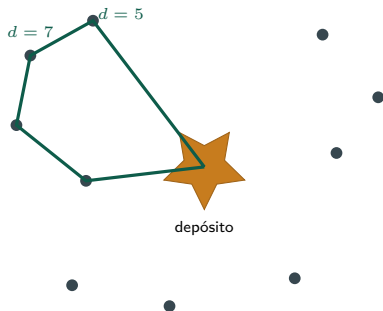
El CVRP en una imagen



Los datos

n clientes con **demanda conocida**, un depósito, y m vehículos idénticos de **capacidad limitada** D .

El CVRP en una imagen



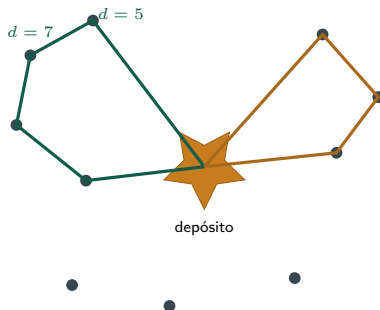
Los datos

n clientes con **demanda conocida**, un depósito, y m vehículos idénticos de **capacidad limitada D** .

La decisión

Diseñar **rutas** que partan y vuelvan al depósito, visiten a **cada cliente exactamente una vez**...

El CVRP en una imagen



Los datos

n clientes con **demanda conocida**, un depósito, y m vehículos idénticos de **capacidad limitada D** .

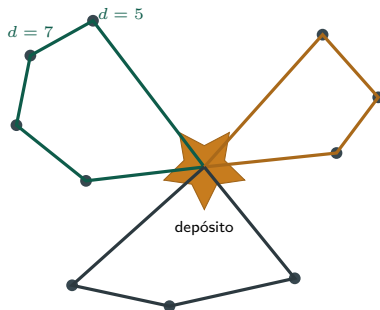
La decisión

Diseñar **rutas** que partan y vuelvan al depósito, visiten a **cada cliente exactamente una vez**...

Las restricciones

... sin que la demanda acumulada de una ruta exceda D , **minimizando la distancia total**.

El CVRP en una imagen



Los datos

n clientes con **demanda conocida**, un depósito, y m vehículos idénticos de **capacidad limitada D** .

La decisión

Diseñar **rutas** que partan y vuelvan al depósito, visiten a **cada cliente exactamente una vez**...

Las restricciones

... sin que la demanda acumulada de una ruta exceda D , **minimizando la distancia total**.

Idea clave

El CVRP es **NP-hard**: combina **particionar** clientes en rutas y **ordenar** las visitas dentro de cada una. Por eso el curso lo aborda dos veces: método **exacto** (Informe 1) y **metaheurística** (Informe 2).

Función objetivo

Minimizar la distancia total recorrida por la flota:

$$\text{mín } z = \sum_{r \in \mathcal{R}} \sum_{\{i,j\} \in r} c_{ij}$$

sujeto a: cada cliente en exactamente una ruta, y

$$\sum_{i \in r} d_i \leq D \quad \forall r \in \mathcal{R}.$$

Instancias (CVRPLIB, Sets A y E)

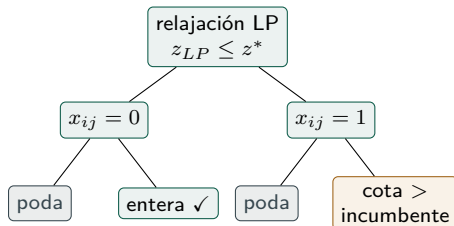
- **5 compartidas** con el Informe 1 ($n = 13$ a 33): comparación directa.
- **4 mayores** ($n = 60$ a 101): escalabilidad, fuera del alcance del exacto.
- Costos: distancia euclidiana redondeada (EUC_2D).

Idea clave

Referencias: **óptimo certificado** donde el exacto lo demostró; **BKS** (mejor solución conocida) en el resto.

Informe 1: método exacto

La idea: Branch and Bound con cortes RCI



Laporte & Nobert (1983)

- Relajación lineal → cota inferior.
- **Ramificar** sobre variables fraccionarias; **podar** ramas dominadas.

Cortes RCI

Los *rounded capacity inequalities* se agregan **iterativamente** al detectar subrutas que violan capacidad: fortalecen la cota sin agrandar el modelo de entrada.

Idea clave

Si el árbol se agota: **óptimo certificado**. Ese certificado es lo que la metaheurística no puede dar.

Resultados del método exacto (Informe 1)

Instancia	n	z	Iter.	Cortes	Tiempo (s)
E-n13-k4	13	247	15	47	0,9
E-n22-k4	22	375	13	84	0,8
E-n23-k3	23	569	16	135	1,2
A-n32-k5	32	784	99	788	576,1
A-n33-k5	33	667 [†]	384	2 846	17 096,4

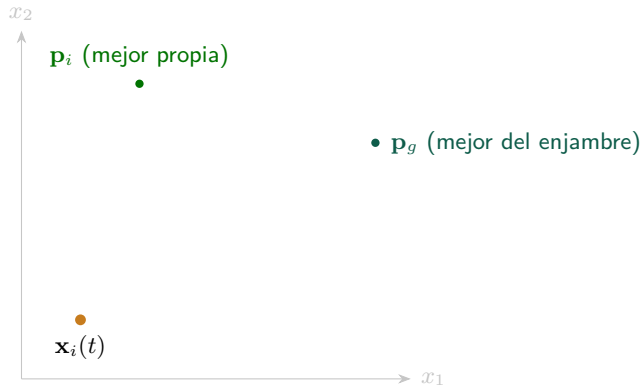
[†] Sin certificado de optimalidad (límite de 60s por resolución del modelo entero); el óptimo real es 661.

Idea clave

La tratabilidad se pierde de forma **abrupta**, no gradual: **un solo cliente más** ($n = 32 \rightarrow 33$) lleva de 10 minutos a **4,7 horas sin certificado**. Más allá de ese tamaño, el exacto queda fuera de alcance práctico.

Informe 2: PSO para el CVRP

PSO en 30 segundos: tres “tirones” que se suman



Actualización canónica

$$\mathbf{v}_i \leftarrow w\mathbf{v}_i + c_1\mathbf{r}_1 \otimes (\mathbf{p}_i - \mathbf{x}_i) + c_2\mathbf{r}_2 \otimes (\mathbf{p}_g - \mathbf{x}_i)$$

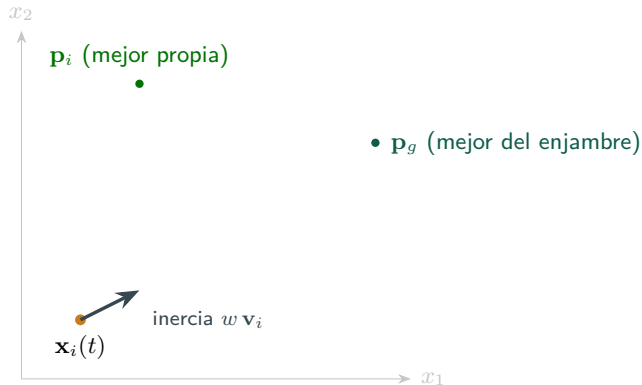
$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$$

- Inercia: sigue el rumbo.
- Cognitivo: hacia su recuerdo.
- Social: hacia el mejor del enjambre.

Idea clave

El progreso emerge de **compartir información**, no de un individuo.

PSO en 30 segundos: tres “tirones” que se suman



Actualización canónica

$$\mathbf{v}_i \leftarrow w\mathbf{v}_i + c_1\mathbf{r}_1 \otimes (\mathbf{p}_i - \mathbf{x}_i) + c_2\mathbf{r}_2 \otimes (\mathbf{p}_g - \mathbf{x}_i)$$

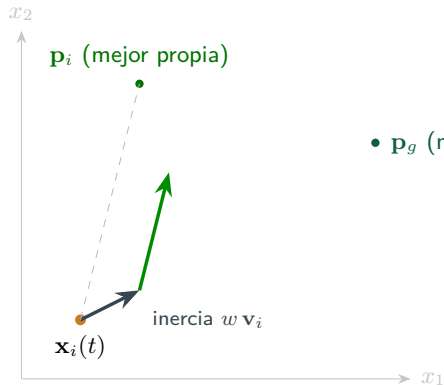
$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$$

- Inercia: sigue el rumbo.
- Cognitivo: hacia su recuerdo.
- Social: hacia el mejor del enjambre.

Idea clave

El progreso emerge de **compartir información**, no de un individuo.

PSO en 30 segundos: tres “tirones” que se suman



- $\mathbf{p}_g$ (mejor del enjambre)

Actualización canónica

$$\mathbf{v}_i \leftarrow w\mathbf{v}_i + c_1\mathbf{r}_1 \otimes (\mathbf{p}_i - \mathbf{x}_i) + c_2\mathbf{r}_2 \otimes (\mathbf{p}_g - \mathbf{x}_i)$$

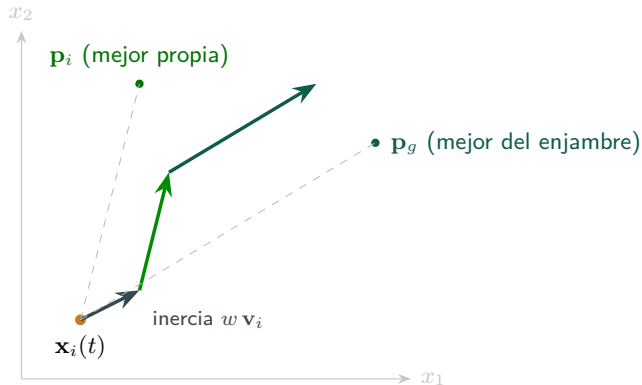
$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$$

- ■ Inercia: sigue el rumbo.
- ■ Cognitivo: hacia su recuerdo.
- ■ Social: hacia el mejor del enjambre.

Idea clave

El progreso emerge de compartir información, no de un individuo.

PSO en 30 segundos: tres “tirones” que se suman



Actualización canónica

$$\mathbf{v}_i \leftarrow w\mathbf{v}_i + c_1\mathbf{r}_1 \otimes (\mathbf{p}_i - \mathbf{x}_i) + c_2\mathbf{r}_2 \otimes (\mathbf{p}_g - \mathbf{x}_i)$$

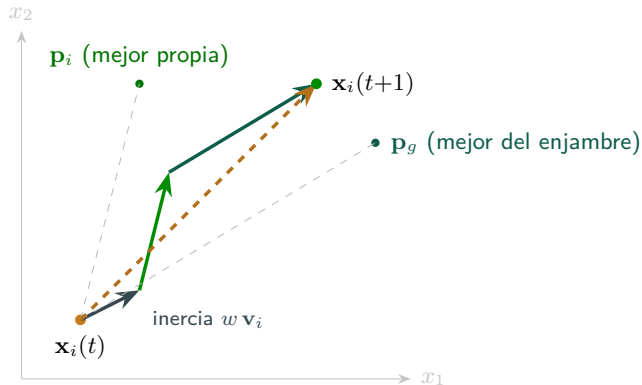
$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$$

- Inercia: sigue el rumbo.
- Cognitivo: hacia su recuerdo.
- Social: hacia el mejor del enjambre.

Idea clave

El progreso emerge de **compartir información**, no de un individuo.

PSO en 30 segundos: tres “tirones” que se suman



Actualización canónica

$$\mathbf{v}_i \leftarrow w\mathbf{v}_i + c_1\mathbf{r}_1 \otimes (\mathbf{p}_i - \mathbf{x}_i) + c_2\mathbf{r}_2 \otimes (\mathbf{p}_g - \mathbf{x}_i)$$

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$$

- Inercia: sigue el rumbo.
- Cognitivo: hacia su recuerdo.
- Social: hacia el mejor del enjambre.

Idea clave

El progreso **emerge de compartir información**, no de un individuo.

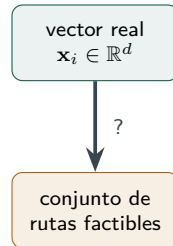
El desafío: PSO es continuo, el CVRP es combinatorio

Lo que PSO sabe hacer

Mover **vectores reales**: $\mathbf{x}_i \in \mathbb{R}^d$, sumar velocidades, interpolar posiciones.

Lo que el CVRP necesita

Un conjunto de **rutas**: no existe una noción natural de “sumar una velocidad” a una partición de clientes.



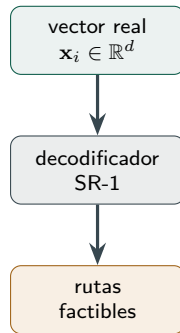
Idea clave

El desajuste entre un algoritmo **continuo** y un problema **combinatorio** es la decisión de diseño central: hay que definir cómo un vector real representa una solución del CVRP y cómo recuperar las rutas a partir de él.

La solución: representación SR-1 (Ai & Kachitvichyanukul, 2009)

La idea

- La partícula **sigue siendo un vector real** de $(n - 1) + 2m$ dimensiones.
- Un **decodificador** la traduce a rutas factibles en capacidad.
- Las ecuaciones de PSO **no se modifican**.



Idea clave

Se descartaron el **PSO discreto** (redefine la velocidad como secuencias de *swaps*) y los enfoques **híbridos** (complejidad adicional que impediría aislar el comportamiento propio de PSO).

Decodificación SR-1 paso a paso (ejemplo del paper: 6 clientes, 2 vehículos)

	claves de prioridad (clientes 1–6)						puntos de ref. (veh. 1 y 2)			
	1	2	3	4	5	6				
Partícula:	0.4	0.2	0.9	0.5	0.6	0.1	0.8	1.5	0.3	1.2

Decodificación SR-1 paso a paso (ejemplo del paper: 6 clientes, 2 vehículos)

	claves de prioridad (clientes 1–6)						puntos de ref. (veh. 1 y 2)			
	1	2	3	4	5	6				
Partícula:	0.4	0.2	0.9	0.5	0.6	0.1	0.8	1.5	0.3	1.2

1. Ordenar claves ascendente (SPV): prioridad = 6, 2, 1, 4, 5, 3

Decodificación SR-1 paso a paso (ejemplo del paper: 6 clientes, 2 vehículos)

	claves de prioridad (clientes 1–6)						puntos de ref. (veh. 1 y 2)			
	1	2	3	4	5	6				
Partícula:	0.4	0.2	0.9	0.5	0.6	0.1	0.8	1.5	0.3	1.2

1. Ordenar claves ascendente (SPV):

prioridad = 6, 2, 1, 4, 5, 3

2. Referencias de vehículos:

$V_1 = (0,8; 0,3)$ $V_2 = (1,5; 1,2) \Rightarrow$ cada cliente prefiere al más cercano

Decodificación SR-1 paso a paso (ejemplo del paper: 6 clientes, 2 vehículos)

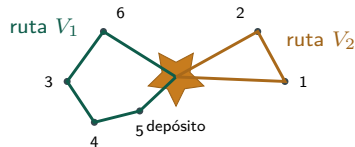
	claves de prioridad (clientes 1–6)						puntos de ref. (veh. 1 y 2)			
	1	2	3	4	5	6				
Partícula:	0.4	0.2	0.9	0.5	0.6	0.1	0.8	1.5	0.3	1.2

1. Ordenar claves ascendente (SPV): $\text{prioridad} = 6, 2, 1, 4, 5, 3$
2. Referencias de vehículos: $V_1 = (0,8; 0,3)$ $V_2 = (1,5; 1,2) \Rightarrow$ cada cliente prefiere al más cercano
3. Insertar en ese orden, al menor costo, respetando capacidad; 2-opt tras cada inserción.

Decodificación SR-1 paso a paso (ejemplo del paper: 6 clientes, 2 vehículos)

	claves de prioridad (clientes 1–6)						puntos de ref. (veh. 1 y 2)			
	1	2	3	4	5	6				
Partícula:	0.4	0.2	0.9	0.5	0.6	0.1	0.8	1.5	0.3	1.2

1. Ordenar claves ascendente (SPV): prioridad = 6, 2, 1, 4, 5, 3
2. Referencias de vehículos: $V_1 = (0,8; 0,3)$ $V_2 = (1,5; 1,2) \Rightarrow$ cada cliente prefiere al más cercano
3. Insertar en ese orden, al menor costo, respetando capacidad; 2-opt tras cada inserción.

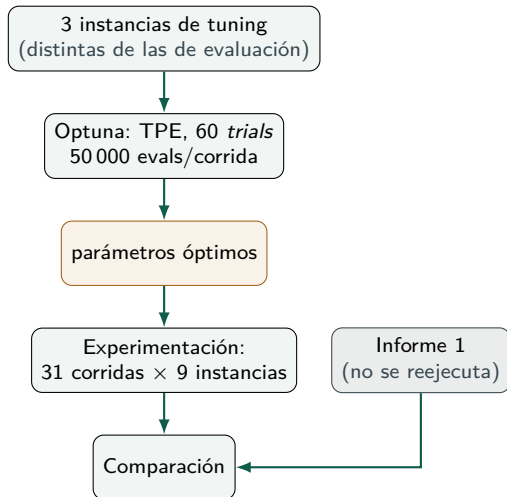


V_1 : 0-6-3-4-5-0

V_2 : 0-2-1-0

► Rutas factibles en capacidad por construcción: en las **279** corridas, ningún cliente quedó sin asignar.

Parametrización con Optuna y diseño experimental



Parámetros encontrados

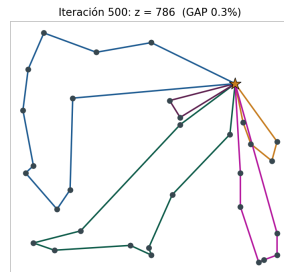
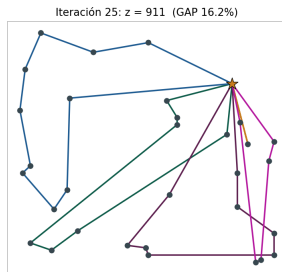
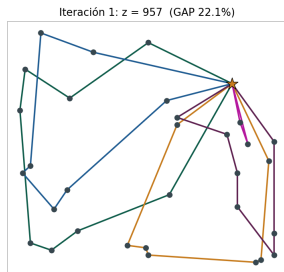
N	partículas	100
T	iteraciones	500
w	inercia	0,91 → 0,28
c_1, c_2	cognitivo/social	2,40 / 1,18
$v_{\text{máx}}$	tope velocidad	0,38

Idea clave

Separar **tuning** de **evaluación** evita el sobreajuste de parámetros. GAP de tuning: 8,54 % promedio.

Resultados y comparación

El enjambre aprendiendo rutas (A-n32-k5, corrida real)



Iteración 1: rutas cruzadas (GAP 22 %).

Iteración 25: lo social ordena sectores (GAP 16 %).

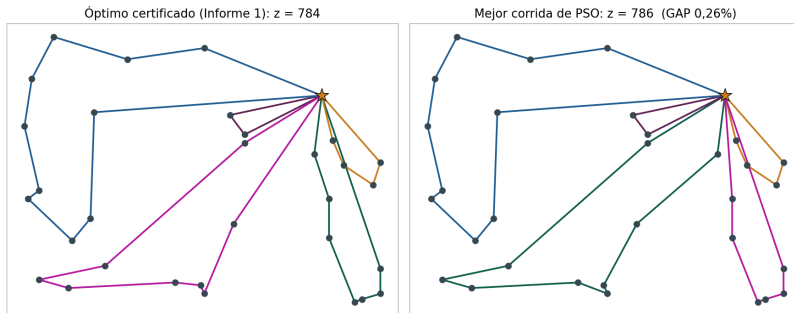
Iteración 500: rutas limpias (GAP 0,26 %).

Idea clave

El mejor global, **decodificado a rutas**, pasa de un ovillo aleatorio a 0,26 % del óptimo en $\approx 0,1$ segundos.

Rutas finales: óptimo certificado vs. mejor PSO

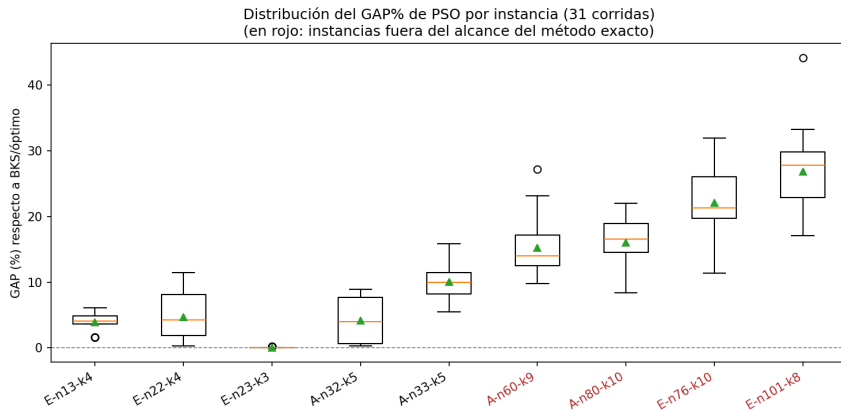
A-n32-k5: rutas del óptimo vs. la mejor solución de PSO



Idea clave

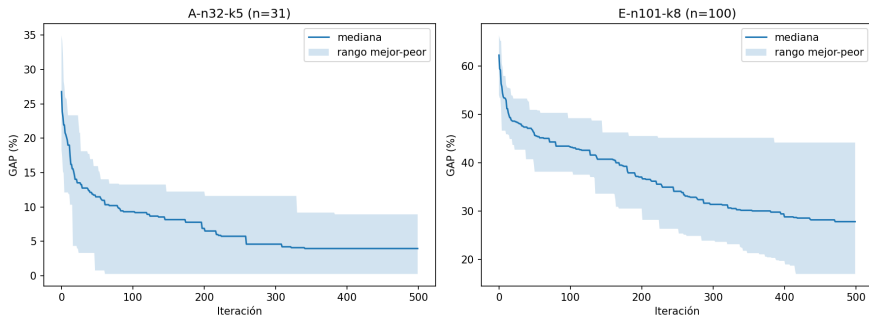
Rutas **casi indistinguibles**: misma sectorización, pequeñas diferencias de orden interno (784 vs. 786, GAP 0,26 %).

Calidad de solución: GAP % por instancia (31 corridas)



Compartidas ($n \leq 33$): GAP promedio 0,03 %–10,07 %, toca el óptimo en E-n23-k3. **Escalabilidad** ($n \geq 60$): 15,25 %–26,84 %, crece al salir del rango de tuning.

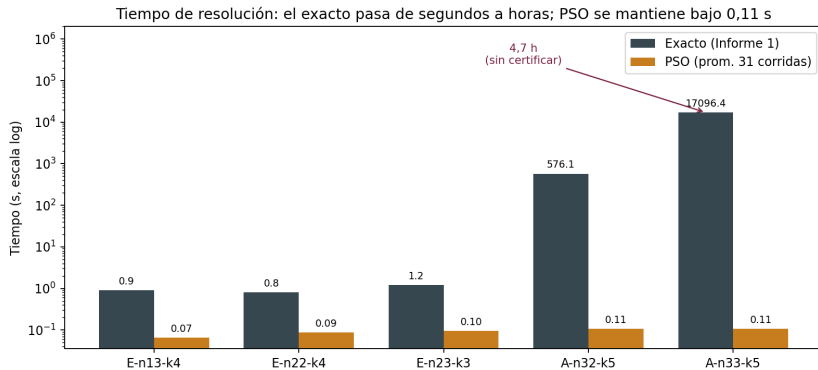
Convergencia: el presupuesto sobra en las chicas, apenas alcanza en las grandes



Idea clave

En A-n32-k5 la mediana deja de mejorar en la iteración **341** de 500; en E-n101-k8 sigue mejorando hasta la **471**.

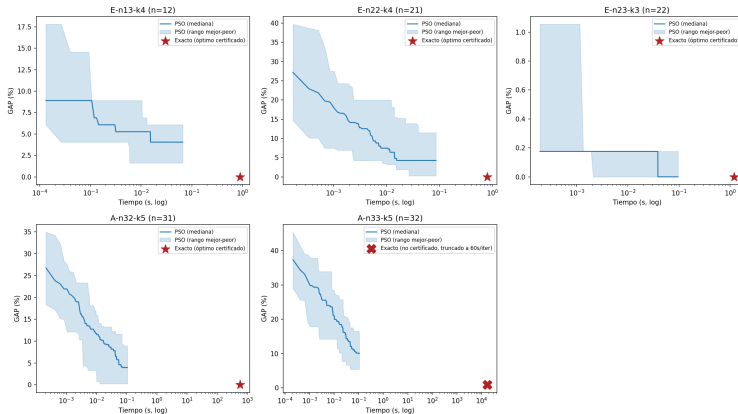
Tiempo de resolución: el acantilado del método exacto



Idea clave

Mismo equipo para ambos métodos. En A-n32-k5 PSO es $\approx 5\,460\times$ más rápido; en A-n33-k5, $\approx 161\,000\times$.

La comparación completa: PSO converge antes de que el exacto despegue



El exacto se reporta como un **punto** (★ óptimo certificado, × truncado sin certificar). Eje de tiempo en **escala log**.

Conclusiones

Conclusiones: los dos métodos, cara a cara

Método exacto

- **Certifica optimalidad:** preferible sin ambigüedad mientras sea tratable.
- Pero la tratabilidad se pierde **de golpe**: $n=32 \rightarrow 33$ pasó de minutos a 4,7 h sin certificado.

PSO (SR-1)

- Sin certificado, pero **tiempo acotado y predecible** ($\leq 0,31$ s hasta $n=101$).
- $GAP \leq 5\%$ cerca del rango de tuning; hasta 27 % al alejarse de él.

Idea clave

La conclusión central es la **complementariedad**, no la sustitución: **el exacto certifica pero no escala; PSO escala pero no certifica.**

Conclusiones: hallazgos específicos de este trabajo

Más allá de la dicotomía general, los datos revelan cuatro observaciones concretas:

- 1 **El acantilado es abrupto, no una pendiente.** Un solo cliente más ($n=32 \rightarrow 33$) multiplicó el tiempo del exacto por ≈ 30 ($576\text{ s} \rightarrow 4,7\text{ h}$) y aun así **no certificó**: la intratabilidad aparece de golpe.
- 2 **Donde ambos aplican, certificar aporta poco a la *calidad*.** En 4 de las 5 instancias compartidas PSO quedó a $< 5\%$ del óptimo real, en *miles* de veces menos tiempo: el certificado del exacto vale por la garantía, no por una mejora tangible de calidad ahí.
- 3 **La dificultad de PSO no depende solo de n .** E-n76-k10 (GAP 22%) resultó más difícil que A-n80-k10 (GAP 16%) pese a ser más chica: pesa la **estructura** de la instancia (razón demanda/capacidad, familia), no solo su tamaño.
- 4 **El presupuesto fijo de evaluaciones es un arma de doble filo.** Sobra en instancias chicas (A-n32-k5 converge en la iteración 341/500) y falta en las grandes (E-n101-k8 aún mejora en la 471/500).

Conclusiones: recomendación y trabajo futuro

Idea clave

Recomendación práctica: usar el **método exacto** mientras la instancia caiga antes del acantilado ($n \lesssim 32$ en este banco), donde es rápido y certifica; a partir de ahí, **PSO** es la única de las dos opciones que devuelve una solución usable en tiempo acotado.

En el informe

Trabajo futuro (derivado directamente de los hallazgos anteriores):

- Variante **GLNPSO** con vecindario social ($lbest/nbest$) para cerrar la brecha de GAP frente a la literatura (hallazgo 2).
- **Escalar** el número de iteraciones con la dimensión $n+2m$, en vez de fijarlo por presupuesto (hallazgo 4).
- Ampliar el **tuning** a todo el rango de tamaños y estructuras evaluadas (hallazgos 3 y 4).

Referencias

Problema y método exacto (Informe 1)

Laporte, G., & Nobert, Y. (1983). A branch and bound algorithm for the capacitated vehicle routing problem. *Operations Research Spektrum*, 5(2), 77–85.

Lysgaard, J., Letchford, A. N., & Eglese, R. W. (2004). A new branch-and-cut algorithm for the capacitated vehicle routing problem. *Mathematical Programming*, 100, 423–445.

Toth, P., & Vigo, D. (Eds.). (2002). *The Vehicle Routing Problem*. SIAM.

CVRPLIB (2024). *Capacitated Vehicle Routing Problem Library*. vrp.atd-lab.inf.puc-rio.br.

Herramientas: Harris et al. (2020), *NumPy*, Nature 585, 357–362; Lam et al. (2015), *Numba*, Proc. LLVM-HPC; Akiba et al. (2019), *Optuna*, Proc. ACM SIGKDD, 2623–2631.

Metaheurística: PSO (Informe 2)

Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proc. IEEE ICNN*, 1942–1948.

Shi, Y., & Eberhart, R. (1998). A modified particle swarm optimizer. *Proc. IEEE CEC*, 69–73.

Clerc, M., & Kennedy, J. (2002). The particle swarm: explosion, stability, and convergence. *IEEE Trans. Evol. Comput.*, 6(1), 58–73.

Poli, R., Kennedy, J., & Blackwell, T. (2007). Particle swarm optimization: An overview. *Swarm Intelligence*, 1(1), 33–57.

Ai, T. J., & Kachitvichyanukul, V. (2009). Particle swarm optimization and two solution representations for solving the CVRP. *Computers & Industrial Engineering*, 56(1), 380–387.

Gracias

¿Preguntas?

Cristopher Angulo Ahumada

Código y datos: github.com/OrejiNet/CVRP-Exact-Method-vs-Particle-Swarm-Optimization

Apéndice: resultados completos de PSO (31 corridas por instancia)

Instancia	n	Ref.	Mejor	Prom.	Desv.	Gap prom. (%)	t (s)
E-n13-k4	13	247	251	256,6	3,0	3,89	0,066
E-n22-k4	22	375	376	392,5	12,3	4,66	0,086
E-n23-k3	23	569	569	569,2	0,4	0,03	0,096
A-n32-k5	32	784	786	816,7	25,4	4,17	0,106
A-n33-k5	33	661	697	727,6	16,2	10,07	0,106
A-n60-k9	60	1354	1486	1560,5	56,3	15,25	0,172
E-n76-k10	76	830	924	1013,2	40,3	22,07	0,212
A-n80-k10	80	1763	1911	2045,4	71,3	16,02	0,226
E-n101-k8	101	815	954	1033,7	44,2	26,84	0,306

Presupuesto: 50 000 evaluaciones por corrida; semillas 0, ..., 30. Núcleo de decodificación acelerado con Numba (~130 000–200 000 evaluaciones/s).

Apéndice: espacio de búsqueda de la parametrización

Parámetro	Rango explorado	Óptimo (Optuna)
N (partículas)	{20, 30, 50, 80, 100}	100
w_0	[0,5, 0,95]	0,9116
w_f	[0,1, 0,5]	0,2782
c_1	[0,5, 2,5]	2,4045
c_2	[0,5, 2,5]	1,1843
$v_{\text{máx}}$ (fracción)	[0,1, 1,0]	0,3762

Muestreador TPE; 60 *trials* ($\approx 10\times$ el número de hiperparámetros); $T = \lfloor 50\,000/N \rfloor$ para igualar esfuerzo entre configuraciones. Instancias de tuning: A-n34-k5 (GAP 3,34 %), A-n62-k8 (7,92 %), E-n76-k7 (14,37 %).